

미니 프롬프트톤 대회 가이드북

1. 좋은 프롬프트의 조건

- 1) **명확한 동사(Action Verb)** 사용
“설명해줘(explain)”, “수정해줘(correct)”, “비교해줘(compare)” 등
- 2) **모호하지 않은 언어** 사용
“적당히” 대신 “표준어 기준으로”
- 3) 한 문장에 하나의 지시만 포함

예시 비교

✗ “이 문장 좀 봐줘.”

✓ “다음 문장의 맞춤법을 모두 교정해줘.”

2. 프롬프트의 다섯 요소

- 1) Instruction 해야 할 일
ex. “맞춤법을 교정하라”
- 2) Context: 배경 정보
ex. “이 문장은 학생이 쓴 일기이다.”
- 3) Example: 예시
ex. “입력: ‘학교가좋아.’ → 출력: ‘학교가 좋아.’”
- 4) Input Data: 실제 데이터
ex. 맞춤법이 틀린 문장들
- 5) Output: 원하는 결과 형식
ex. 수정된 문장만 출력

- 좋은 프롬프트 예시

(Instruction)

너는 고등학교 국어 교사이다. 학생이 쓴 일기문을 검토하여 맞춤법과 띄어쓰기만 교정하라.
문장의 의미는 바꾸지 말고, 수정된 문장만 출력하라.

(Context)

이 문장은 학생이 일기 과제로 제출한 글이다.
따뜻하고 자연스러운 어투를 유지하되, 문법적 오류는 모두 바로잡아야 한다.

(Example)

입력: 어제는진짜힘든하루였다그래도버텼어
출력: 어제는 진짜 힘든 하루였다. 그래도 버텼어.

(Input Data)

입력: 내일은학교 끝나고친구들이랑놀러가야지

(Output)

출력:

3. 예시 기반 프롬프트 설계

모델은 예시를 통해 학습된 형식을 따르는 성향이 있다.

3.1 Zero-Shot

> “이 문장의 맞춤법을 고쳐줘.”

3.2 One-Shot

> 입력과 결과를 1개 제공

- 예시

입력: “난 학교가 좋아.”

출력: “난 학교가 좋아.”

이제 다음 문장을 고쳐줘: “오늘은비가온다.”

#5.3 Few-Shot

> 여러 예시를 제공

> 예시 3~5개를 주면 모델이 패턴을 안정적으로 학습한다.

4. 역할(Role)과 스타일 설정

역할을 부여하면 결과 품질이 크게 향상된다.

- 역할 부여 예:

“너는 국어 교사야. 학생의 문장을 맞춤법에 맞게 고쳐라.”

- 스타일 지정:

“공식 문체로, 문장부호를 유지해라.”

5. 구조적 프롬프트 설계

구조적 프롬프트는 지시·입력·출력 구역을 명확히 구분해 모델이 해야 할 일과 처리할 데이터를 혼동하지 않게 만든다. 이는 결과의 일관성과 정확도를 높여 불필요한 설명이나 오류 출력을 줄여준다.

- 예시

지시

다음 문장의 맞춤법을 교정하라.

입력

나는밥을먹었다

출력

나는 밥을 먹었다

6. 사고 유도형 프롬프트 (Reasoning Prompt)

모델이 “생각의 과정”을 말하게 하면 정확도 ↑

> “단계별로 생각해서 맞춤법 오류를 찾고 수정 과정을 설명한 뒤, 최종 문장을 출력하라.”

디버깅에도 유리하다.

7. 환각(Hallucination) 방지

모델이 틀린 사실을 “지어내지 않도록” 다음을 지시한다.

“모르겠으면 ‘모르겠습니다’라고 말하라.”

“사실과 의견을 구분하라.”

부록: 추천 프롬프트 예시

[시스템]

너는 국어 교사이다. 문장의 맞춤법과 띄어쓰기를 정확히 수정하라.

[사용자]

입력

나는 학교가 좋아

출력

나는 학교가 좋아